MODULE 07 · DEVOPS BOOTCAMP

Infrastructure as Code with Terraform

Stop clicking. Start describing. Then let a machine make reality match the description — every time, in every region, for everyone on the team.

`terraform` `aws` `kubernetes` `hcl` `3-hour lab`

Concepts first — the keyboard comes after the break.

01

The Problem With Clicking

Every console click is a decision nobody wrote down. Multiply that by six months and a team of five, and nobody can say what your infrastructure actually is.

clickops · snowflakes · drift · tribal knowledge

THE HONEST VERSION

How infrastructure gets built by hand

1

Someone clicks

A senior engineer builds the VPC, the subnets, the security groups. It works. It ships.

2

Someone writes a wiki page

Fourteen screenshots. Accurate for about three weeks.

3

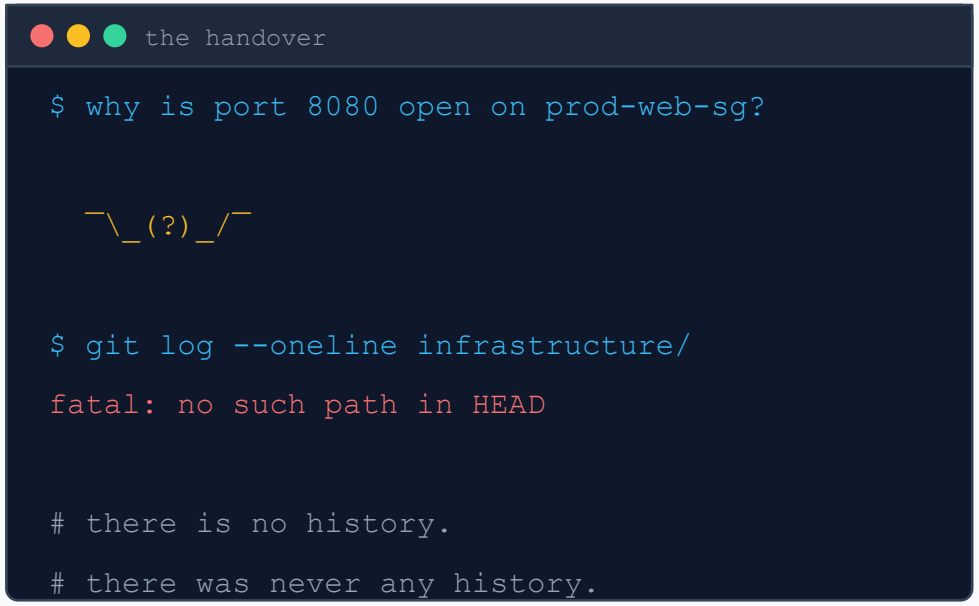
Someone else hotfixes it

A port is opened at 2am during an incident. The wiki is not updated.

4

Someone leaves

The only person who knew why that security group rule exists is now at another company.



THE REAL COST

None of these steps is wrong on its own. The failure is cumulative: infrastructure becomes an oral tradition, and oral traditions do not survive staff turnover, audits, or outages.

Ask the room: who has inherited a system nobody could explain? Nearly every hand goes up.

FAILURE MODES

Four ways manual infrastructure fails you

Snowflake servers

Every box is subtly unique. Staging has a package prod doesn't. The bug reproduces on exactly one machine, and nobody knows why.

Configuration drift

Reality slowly walks away from the documentation. Nobody notices until the thing you're restoring from doesn't match the thing you lost.

No audit trail

Who opened that port? When? For which ticket? CloudTrail tells you an API call happened. It does not tell you the intent behind it.

Can't reproduce

"Build me an identical staging environment" becomes a two-week project instead of a twenty-minute one.

These four are why IaC exists. Every Terraform feature we cover today is an answer to one of them.

THE STRESS TEST

Two questions that break a manual setup

“The Mumbai region is down.

Stand everything up in Singapore.”

By hand: a day of clicking, if you still remember every setting. Half your security groups will be subtly wrong.

“An auditor is here.

Prove prod matches what was approved.”

By hand: you screenshot the console and hope. There is no approved version to compare against.

with infrastructure as code

```
$ terraform apply -var region=ap-southeast-1
Apply complete! Resources: 34 added, 0 changed, 0 destroyed.

$ git log --oneline -- infra/prod/     # every change, every reviewer, every reason
a3f9c21  open 8080 for the payments spike  (reviewed-by: sita)  #INC-412
```

Same two questions. One is a day of anxiety; the other is a command and a git log.

THE SIX PROPERTIES

What we actually want from infrastructure

Reproducible

Same input, same infrastructure. Every time.

Versioned

It lives in git. You can diff it and roll it back.

Reviewable

Changes go through a pull request, like code.

Auditable

Who, what, when and why — from the commit log.

Disposable

Tearing it all down is one safe, complete command.

Self-documenting

The config is the documentation. It cannot go stale.

Notice that four of the six come free the moment infrastructure becomes a text file in a repository.

THE DEFINITION

Infrastructure as Code, defined

Managing and provisioning infrastructure through machine-readable definition files, rather than through interactive configuration tools or physical hardware setup.

Definition files

Text. In a repository. Human-readable, diffable, reviewable — no binary blobs, no screenshots, no wiki.

Machine-readable

A tool reads the files and makes the API calls. Humans state the goal; the machine does the clicking.

Provisioning

Not just servers. Networks, IAM roles, DNS records, buckets, clusters, alerts, dashboards — all of it.

THE ONE-LINE VERSION

If you cannot rebuild your entire environment from a git clone and one command, you do not have Infrastructure as Code — you have documentation that happens to be executable in places.

02

Ways to Say It

Before the syntax: four distinctions that decide which tool you reach for, and which mistakes you are about to make.

declarative idempotent immutable provisioning

THE CORE DISTINCTION

Imperative vs declarative

IMPERATIVE — you describe the steps

create-bucket.sh

```
#!/usr/bin/env bash
if aws s3api head-bucket \
     --bucket $NAME 2>/dev/null; then
  echo "exists, skipping"
else
  aws s3api create-bucket \
     --bucket $NAME --region ap-south-1
fi
# + versioning, tags, policy, ...
```

You own every branch. Every “does it already exist?” check is your problem, forever.

DECLARATIVE — you describe the destination

main.tf

```
resource "aws_s3_bucket" "data" {
  bucket = var.bucket_name
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration {
    status = "Enabled"
```

You state the end result. Terraform works out whether that means create, update, or do nothing at all.

WHY IT MATTERS

The imperative script grows a new branch for every edge case until nobody dares run it. The declarative file stays the same size as your infrastructure — because it describes what you want, not the path to get there.

THE PROPERTY THAT MAKES IT SAFE

Idempotency: run it again, nothing happens

An operation is idempotent when running it a second time changes nothing. It is what lets you re-run a deployment at 3am without first working out how far the last one got.

first run

```
$ terraform apply

  # aws_s3_bucket.data will be created
  + resource "aws_s3_bucket" "data" {
      + bucket = "chiya-data-sagyam"
    }

Apply complete! 1 added, 0 changed, 0 destroyed.
```

second run — and the fiftieth

```
$ terraform apply

No changes. Your infrastructure matches
the configuration.

Apply complete! 0 added, 0 changed,
0 destroyed.
```

THE PRACTICAL PAYOFF

You never have to remember what state a deployment was left in. Ran halfway and crashed? Run it again. Not sure if a colleague already applied? Run it again. “Run it again” is always the safe answer.

PETS AND CATTLE

Mutable vs immutable infrastructure

Mutable — upgrade in place

SSH in, patch the package, restart the service. The server keeps its identity and accumulates history.

After a year, no two servers have had the same sequence of changes applied. This is how snowflakes are born.

Immutable — replace, don’t repair

Build a new image, launch new instances from it, shift traffic, delete the old ones. Nothing is ever modified after creation.

Rollback is trivial: the previous version still exists and still works.

You have already been doing this

A container image is immutable infrastructure. You never patch a running container — you build a new image, push a new tag, and roll out. Terraform brings that same instinct up a level, to the servers, networks and clusters underneath.

`docker build` `push :v2` `kubectl rollout` `terraform apply`

Terraform makes replacement cheap enough to prefer. Cheap replacement is what makes immutability practical.

TERRAFORM AND ANSIBLE ARE NOT COMPETITORS

Provisioning vs configuration management

Provisioning

Creates the things that did not exist.

- VPCs, subnets, security groups
- EC2 instances, EBS volumes, load balancers
- IAM roles, policies, OIDC providers
- S3 buckets, ECR repositories, EKS clusters
- DNS records, certificates, alarms

Configuration management

Shapes what is inside the thing.

- Installing and pinning packages
- Writing config files from templates
- Managing users, groups and permissions
- Enabling and restarting systemd services
- Deploying application artefacts

HOW THEY FIT TOGETHER

Terraform creates the EC2 instance; Ansible configures what runs on it. In practice the seam is moving: with containers, the “inside” is baked into an image at build time, so Terraform provisions the cluster and the image carries the configuration.

Both tools overlap at the edges. Overlap is not a reason to pick one and force it to do everything.

KNOW WHAT ELSE IS OUT THERE

The IaC tool landscape

TOOL	SCOPE	LANGUAGE	PICK IT WHEN
Terraform	Multi-cloud	`HCL (declarative)`	You want the default answer and the biggest ecosystem
OpenTofu	Multi-cloud	`HCL (declarative)`	You need a truly open-source licence
CloudFormation	AWS only	`YAML / JSON`	You are all-in on AWS and want native support
AWS CDK	AWS only	`TypeScript, Python…`	Your team would rather write real code than HCL
Pulumi	Multi-cloud	`TypeScript, Go, Python`	You want loops and types from a general language
Bicep	Azure only	`Bicep DSL`	You are on Azure and escaping ARM templates
Crossplane	Multi-cloud	`Kubernetes YAML`	Your control plane is already Kubernetes

WHY WE TEACH TERRAFORM

Not because it is the best on every axis — CDK has nicer abstractions and CloudFormation has deeper AWS integration. Because it is the one you are most likely to walk into on your first day, it works across every provider, and the concepts transfer directly to all the others.

The ideas today — declarative config, state, plan-then-apply, modules — are shared by every tool in this table.

TERRAFORM, THE BSL, AND OPENTOFU

One thing you will be asked in an interview

In August 2023 HashiCorp changed Terraform's licence from the Mozilla Public Licence to the Business Source Licence. The community forked the last open version as OpenTofu, now a Linux Foundation project. IBM completed its acquisition of HashiCorp in 2025.

Terraform (BSL)

Free for essentially all normal use. The licence restricts building a competing commercial product on it. Largest provider ecosystem, HCP Terraform for teams.

OpenTofu (MPL 2.0)

Drop-in fork. Same HCL, same providers, same state format. Swap terraform for tofu on the command line and most projects keep working.

WHAT THIS MEANS FOR YOU TODAY

Nothing. Everything in this module — HCL syntax, state, plan and apply, modules, the AWS provider — is identical in both tools. Learn the concepts; the binary is a swap. Know the story anyway, because it is a common interview question and it is a useful lesson in what open-source licensing risk actually costs a company.

Verify the current licensing and release status before you quote it in an interview — this space keeps moving.

03

The Language

HCL has exactly one shape — a block with a type, some labels and some arguments.

Learn that shape once and the rest is vocabulary.

`provider` `resource` `data` `variable` `output`

BLOCK ANATOMY

Every line of HCL is one of these

main.tf

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"

  tags = {
    Name = "web-${var.student_name}"
  }
}
```

The local name is yours

"web" never appears anywhere in AWS. It is how you refer to this resource from elsewhere in your config:

aws_instance.web.id

Name things for their role, not their type — api, not instance1.

BLOCK TYPE	What kind of thing this is. resource, data, variable, output, module, provider, terraform.
LABELS	For a resource: the provider type first (aws_instance), then a local name you choose (web).
ARGUMENTS	key = value. Strings, numbers, booleans, lists, maps, and references to other resources.
NESTED BLOCKS	Some arguments are themselves blocks: tags, ingress, versioning_configuration, root_block_device.

BLOCK TYPE 1 OF 5

Providers: the plugin that speaks the API

Terraform core knows nothing about AWS. A provider is a downloaded plugin that translates your HCL into API calls for one platform — and there are thousands of them.

versions.tf

```
terraform {
  required_version = ">= 1.9.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"    # any 5.x, never 6.x
    }
  }
}
```

init downloads them
They land in .terraform/ and pin to .terraform.lock.hcl. Commit that lock file.

Pin your versions
~> 5.0 accepts 5.9 but refuses 6.0. Major versions break things.

default_tags is free policy
Every taggable resource this provider makes gets these tags automatically.

Today you will use three providers: aws to build the cloud, kubernetes to deploy into it, and random to generate unique suffixes.

hashicorp/aws hashicorp/kubernetes hashicorp/random + 4,000 more

default_tags { tags = { ManagedBy = "terraform" } }

BLOCK TYPE 2 OF 5

Resources: the things you own

s3.tf

```
resource "aws_s3_bucket" "data" {
  bucket = "chiya-data-${var.student_name}"
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id
  versioning_configuration {
    status = "Enabled"
```

A resource is a promise

“I want exactly one of these to exist, configured this way, and I want Terraform to be responsible for it.”

That responsibility is the important half. Terraform will create it, update it when you change the file, and destroy it when you delete the block.

THE SURPRISE EVERYONE GETS

AWS features you think of as “settings on a bucket” are often separate resources: versioning, encryption, public-access block, lifecycle rules, and the bucket policy are five more blocks.

This is not Terraform being awkward. It mirrors how the AWS API is actually shaped — each of those is its own API call.

HOW TO FIND THE RIGHT BLOCK

Do not guess and do not trust an LLM’s memory. Search the provider registry for the AWS resource name, read the Example Usage block, and copy from there.

registry.terraform.io is the only source of truth for arguments and their current names.

BLOCK TYPE 3 OF 5

Data sources: read, don’t own

A data block looks up something that already exists. Terraform reads it, uses it, and never manages or deletes it.

data.tf

```
# Never hardcode an AMI ID. They differ per region
# and go stale every few weeks.
data "aws_ssm_parameter" "al2023" {
  name = "/aws/service/ami-amazon-linux-latest/..."
}

data "aws_caller_identity" "me" {}
data "aws_region" "current" {}
```

RESOURCE VS DATA — THE TEST

Would deleting this block delete something in AWS? If yes, it is a resource and you own it. If no, it is a data source and you are only borrowing it.

Look up a moving target

The latest AMI, the current account ID, the availability zones in this region.

Consume someone else’s work

The networking team owns the VPC. You read it by tag and build inside it.

Read a secret at plan time

Pull a value from Secrets Manager or SSM instead of committing it to the repo.

Data sources are also how two separate Terraform projects talk to each other without sharing state.

BLOCK TYPE 4 OF 5

Variables: the knobs on the outside

variables.tf

```
variable "region" {
  description = "AWS region for all lab resources."
  type        = string
  default     = "ap-south-1"    # Mumbai
}

variable "student_name" {
  type = string

  validation {
    condition     = can(regex("^[a-z0-9-]{3,20}$",
```

HOW A VALUE GETS IN — later wins

1. `default in the block` — the fallback
2. `terraform.tfvars` — your local values, gitignored
3. `-var-file=prod.tfvars` — per-environment values
4. `TF_VAR_region=…` — how CI passes values in
5. `-var region=…` — the manual override

TWO HABITS THAT PAY FOR THEMSELVES

Always write a description — it appears in error messages and in generated docs. Always add validation for anything with a format constraint, because a validation error at plan time costs you three seconds, and the same mistake caught by the AWS API costs you a half-applied change to untangle.

BLOCK TYPE 5 OF 5, PLUS A HELPER

Outputs and locals

output — values that leave the module

locals — naming a repeated expression

outputs.tf

```
output "cluster_endpoint" {
  value = aws_eks_cluster.main.endpoint
}

output "db_password" {
```

locals.tf

```
locals {
  prefix = "chiya-${var.env}-${var.student_name}"

  common_tags = {
    Env     = var.env
```

Outputs are how a module reports back, how you find the URL you just created, and how CI passes a value to the next job.

A local is computed once and reused. Use it when the same expression appears three times — not as a substitute for a variable.

SENSITIVE IS NOT SECRET

sensitive = true only stops the value printing to your terminal. It is still stored in plain text in the state file. We will come back to this — it matters more than anything else on this slide.

REFERENCES AND INTERPOLATION

How resources wire together

the reference is the wire

```
resource "aws_iam_role" "ec2" {
  name               = "web-role-${var.student_name}"
  assume_role_policy = data.aws_iam_policy_document.trust.json
}

resource "aws_iam_instance_profile" "ec2" {
  role = aws_iam_role.ec2.name
}
```

The address format

<type>.<name>.<attribute>

```
aws_iam_role.ec2.name

data sources get a data. prefix:
data.aws_ami.al2023.id

variables and locals are flat:
var.region      local.prefix
```

WHY YOU NEVER PASTE AN ID

Writing role = "web-role-sagyam" as a literal string would work — once. Referencing aws_iam_role.ec2.name does two extra things: Terraform learns that the profile depends on the role and must build it second, and if you ever rename the role, every reference follows automatically. Every hardcoded ID is a dependency Terraform cannot see.

ORDER OF OPERATIONS

Terraform builds a graph, not a script

Terraform reads all your files, resolves every reference, and builds a directed graph. Independent branches run in parallel; dependent resources wait. You never write the order.

the bottom branch has no dependents — it starts at the same instant as the top one

Implicit — preferred

A reference creates the edge. This is how 95% of your dependencies should be expressed.

depends_on — the escape hatch

For real dependencies with no reference, e.g. an IAM policy that must attach before an instance can boot.

TWO THINGS THIS BUYS YOU

Speed — an apply that would take twenty minutes in sequence finishes in five, because Terraform runs every independent branch at once. And correctness on the way out: terraform destroy walks the same graph backwards, so nothing is deleted while something still depends on it.

04

The Loop

Four commands, and you will type three of them a hundred times today. The discipline is not in knowing them — it is in reading the plan before you say yes.

`init` `plan` `apply` `destroy`

FOUR COMMANDS

The core workflow

1

`terraform init`

Prepare the directory

Downloads providers, writes the lock file, configures the backend. Run it once per project, and again whenever providers or the backend change.

2

`terraform plan`

Show me the diff

Reads your config, refreshes real state, and prints exactly what it would do. Changes nothing. This is the safest command in the tool.

3

`terraform apply`

Make it so

Re-plans, asks for confirmation, then walks the graph making API calls. This is the only command that costs money.

4

`terraform destroy`

Take it all down

Walks the graph backwards and deletes everything in state. The reason a lab costs cents instead of a month's rent.

a whole session

```
$ terraform init      # once per project
$ terraform plan      # every single time, and actually read it
$ terraform apply     # type yes only after reading the plan
```

THE FOUR SYMBOLS

Reading a plan

```
terraform plan

Terraform will perform the following actions:

  # aws_s3_bucket.data will be created
  + resource "aws_s3_bucket" "data" {
      + bucket = "chiya-data-sagyam"
      + arn    = (known after apply)
    }

  # aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
      ~ ami = "ami-0abc" -> "ami-0def"
```

+ **create**
New resource. Safe.

~ **update in place**
Changed without rebuilding. Usually safe.

-/+ **destroy and recreate**
The old one dies. Read this line every time.

- **destroy**
It goes away. Data goes with it.

BUILD THE HABIT NOW

Read the summary line first: added / changed / destroyed. If the destroyed count is not what you expected, stop and find out why before typing yes. “(known after apply)” is normal — Terraform genuinely cannot know an ARN before AWS assigns it.

THE LINE THAT RUINS AFTERNOONS

forces replacement

Some arguments cannot be changed in place. Editing one silently means delete-and-rebuild.

HYGIENE

The commands around the loop

COMMAND	WHAT IT DOES	WHEN YOU RUN IT
`terraform fmt`	Rewrites your files to canonical style	Before every commit. Add it to pre-commit
`terraform validate`	Checks syntax and internal consistency	In CI, before plan. No AWS access needed
`terraform show`	Prints the current state, human-readable	When you want to see what you own
`terraform output`	Prints the output values	To grab a URL or an ARN for the next lab
`terraform state list`	Lists every resource address in state	When you have lost track of what exists
`terraform console`	A REPL for evaluating expressions	When an interpolation is not doing what you meant

the pre-commit habit

```
$ terraform fmt -recursive && terraform validate
main.tf
Success! The configuration is valid.
```

Why fmt is not bikeshedding

Canonical formatting means a pull-request diff shows only real changes, not somebody's indentation preference. It is the cheapest code-review improvement you will ever make.

RECONCILIATION — YOU HAVE SEEN THIS BEFORE

Desired state, actual state, and the diff between them

Desired state

Your .tf files. What you say you want. Lives in git, reviewed by humans.

Actual state

What is really running in AWS right now, whatever anyone did to it.

The plan

The diff between the two, plus the set of API calls that would close it.

This is the Kubernetes reconciliation loop

A Deployment declares three replicas; a controller watches, notices two, and starts one. Terraform is the same idea with the loop taken out: it reconciles once, when you run apply, and then stops. That single difference explains almost everything about how the two tools behave.

SO NOBODY IS WATCHING

If someone changes a security group in the console, Terraform does not notice or care until the next time a human runs plan.

WHICH IS WHY TEAMS AUTOMATE IT

Run plan on a schedule and alert on any non-empty diff. Now drift is a notification instead of an archaeology project.

05

State Is the Whole Ballgame

Everything else in Terraform is syntax you can look up. State is the concept that decides whether you have a good week or a bad one.

`terraform.tfstate` `drift` `backends` `locking`

THE MAPPING NOBODY TELLS YOU ABOUT

What state actually is

Your config says “a bucket named chiya-data-sagyam.” AWS says “resource arn:aws:s3:::chiya-data-sagyam.” State is the file that remembers those are the same thing — and that Terraform is the one responsible for it.

terraform.tfstate (JSON, abridged)

```
{
  "resources": [{
    "type": "aws_s3_bucket",
    "name": "data",              <- your local name
    "instances": [{
      "attributes": {
        "id":  "chiya-data-sagyam",   <- the real one
        "arn": "arn:aws:s3:::chiya-data-sagyam",
        "region": "ap-south-1"
```

State is why deleting a block from your .tf file deletes the resource, rather than simply forgetting about it.

NOTE THE ATTRIBUTES

State stores every attribute of every resource — including the ones you marked sensitive.

To know what it owns

Without state, Terraform cannot tell a resource it created from one that was always there.

To compute the diff

plan compares config against state, then refreshes state against reality.

To remember the graph

Dependencies are recorded, so destroy can run in the correct reverse order.

IN ORDER OF HOW MUCH THEY WILL HURT YOU

Three rules of state

1

State is the truth about what Terraform owns — not about what exists

Delete a bucket in the console and Terraform still believes it exists, until the next refresh. Create one by hand and Terraform will never touch it. Ownership is recorded in the file, not inferred from AWS.

2

State contains secrets, in plain text, always

Database passwords, private keys, generated tokens. sensitive = true hides them from your terminal, not from the file. Never commit state to git. Encrypt it at rest and restrict who can read the bucket.

3

Two people applying against one state at the same time will corrupt it

Both read the same starting point, both write their result, and one silently overwrites the other. This is the problem locking exists to solve, and it is why local state does not survive a team.

If you remember one slide from this module, make it this one.

AND HOW PLAN CATCHES IT

Drift: when reality walks away

Drift is any difference between what your config describes and what actually exists. Somebody clicked something. An autoscaler changed a value. A service was upgraded underneath you.

you will do exactly this in lab 01

```
$ aws s3 rm s3://chiya-data-sagyam/hello.txt      # change AWS behind Terraform's back
delete: s3://chiya-data-sagyam/hello.txt


$ terraform plan
Note: Objects have changed outside of Terraform


  # aws_s3_object.hello has been deleted
  + resource "aws_s3_object" "hello" {    # will be re-created
```

WHY THIS IS THE WHOLE POINT

Your config is the desired state; AWS holds the actual state; plan is the diff and apply is the correction. Drift is not an error condition — it is the normal condition, and reconciliation is how you keep answering it.

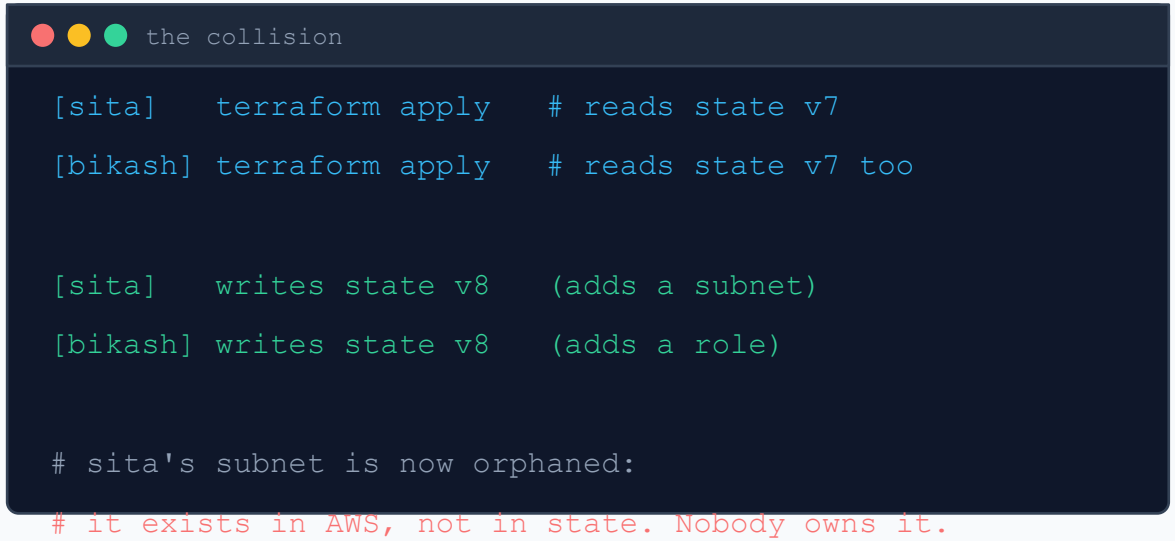
WHY BACKENDS EXIST

Local state fails the moment there are two of you

Your laptop

terraform.tfstate sits in the working directory. It works perfectly — for exactly one person on exactly one machine.

Your colleague clones the repo, runs plan, sees an empty state, and Terraform proposes creating everything a second time.



Shared

State lives in S3, not on a laptop. Everyone reads the same file.

Locked

One apply at a time. The second person waits instead of colliding.

Encrypted

At rest by default, with bucket policies controlling who can read it.

Versioned

Bucket versioning gives you a rollback if state is ever damaged.

A remote backend is not an advanced feature. It is the first thing you set up on any project with more than one contributor.

REMOTE STATE IN PRACTICE

The S3 backend, and its chicken-and-egg

backend.tf

```
terraform {
  backend "s3" {
    bucket       = "tf-state-sagyam-a91c"
    key          = "lab01-s3/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true   # native S3 locking, TF 1.11+
```

migrating existing state

```
$ terraform init -migrate-state
Do you want to copy existing state to the new backend?
  yes
Successfully configured the backend "s3"!
```

The key is a path inside the bucket. One bucket holds state for many projects — give each its own key.

WHY var.bucket DOES NOT WORK HERE

The backend block is read before Terraform evaluates variables, locals or anything else — it has to know where state lives before it can load state.

This trips up literally everyone, exactly once. The professional workaround is partial configuration:

terraform init \

use_lockfile replaced a whole table

Locking used to require provisioning a DynamoDB table. Since 1.11 S3 handles it natively with a .tflock object beside your state.

STATE SURGERY

When state and reality disagree

COMMAND	WHAT IT DOES	THE SITUATION
`state list`	Every resource address Terraform owns	“What do I actually have?”
`state show ADDR`	Every recorded attribute of one resource	“What does it think the ID is?”
`state mv OLD NEW`	Renames an address without touching AWS	You renamed a block and don’t want a rebuild
`state rm ADDR`	Forgets a resource; leaves it alive in AWS	Handing ownership to another project
`import ADDR ID`	Adopts an existing AWS resource into state	Bringing click-built infrastructure under control
`taint / -replace`	Marks a resource for destroy-and-recreate	It exists but it is broken

NEVER HAND-EDIT THE FILE

It is JSON and it looks editable. Editing it desynchronises the serial number and can corrupt state for everyone. Always go through the state commands.

IMPORT IS THE MIGRATION PATH

You will rarely start from an empty account. import (or an import block, since 1.5) is how you take an existing click-built estate and bring it under Terraform one resource at a time.

06

Beyond One File

Everything so far fits in a single directory. Real projects have environments, teams and pipelines — here is the shape they take.

modules

environments

kubernetes

ci/cd

REUSE

Modules: a function call for infrastructure

A module is just a directory of .tf files. Variables are its parameters, outputs are its return values, and calling it twice gives you two copies with different arguments.

calling a module

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"          # pin registry modules too

  name = "chiya-${var.env}"
  cidr = "10.0.0.0/16"
}
```

The root module

The directory you run terraform in is already a module. You have been writing one all along.

Local modules

source = "./modules/network" — start here. Extract a module when you have copied the same block twice.

DON'T MODULARISE TOO EARLY

A module written before you have two real callers is a guess about the future, and it usually guesses wrong. Duplication is cheaper than the wrong abstraction.

A GOOD MODULE HAS A JOB

"A VPC with public and private subnets across three AZs" is a module. "Our infrastructure" is not — that is a root configuration wearing a module's clothes.

LAYOUT

One config, many environments

directory per environment (recommended)

```
infra/
├── modules/
│   ├── network/
│   └── cluster/
├── dev/
│   ├── main.tf          # calls the modules
│   ├── backend.tf       # key = dev/terraform.tfstate
│   └── terraform.tfvars
└── prod/
    ├── main.tf
    ├── backend.tf       # key = prod/terraform.tfstate
```

Separate directories, separate state

Prod has its own state file, its own backend key, and its own IAM credentials. A mistake in dev cannot reach it.

Workspaces: use with care

terraform workspace gives one config several states. Fine for short-lived feature stacks; a poor fit for dev-vs-prod, because one wrong command applies prod config from your laptop.

THE RULE THAT MATTERS MORE THAN THE LAYOUT

Blast radius. Split state along the lines you want failures to stop at — by environment first, then by lifecycle. Networking that changes twice a year does not belong in the same state file as an application that ships twice a day.

TWO PROVIDERS, ONE APPLY

Terraform meets Kubernetes

The aws provider builds the cluster. The kubernetes provider deploys into it. Both in the same configuration — which creates a subtle ordering problem.

wiring one provider to another's output

```
provider "kubernetes" {
  host = aws_eks_cluster.main.endpoint
  cluster_ca_certificate = base64decode(
    aws_eks_cluster.main.certificate_authority[0].data)

  exec {                        # short-lived token
    api_version = "client.authentication.k8s.io/v1beta1"
    command     = "aws"
```

THE CHICKEN AND EGG

Provider configuration is evaluated early. If the cluster does not exist yet, there is no endpoint to point at — and the error message will not say so clearly.

THE FIX

Split it: apply the cluster, then apply the workloads from a second configuration.

Should you deploy applications with Terraform at all?

For cluster-level plumbing that must exist before anything else — namespaces, the ingress controller, cluster roles — yes. For your actual application, usually no: Helm and a GitOps controller like Argo CD deploy far more often than infrastructure changes, and they reconcile continuously. Draw the line at “things the cluster needs” versus “things that run on the cluster.”

WHERE IT ACTUALLY RUNS IN A REAL TEAM

Terraform in a pipeline

1

Open a PR

Config change lands on a branch.

2

fmt + validate

Fast checks. No credentials needed.

3

plan

Posted as a PR comment for review.

4

Human approves

The plan is the diff being reviewed.

5

apply on merge

Only from main. Only from CI.

.github/workflows/terraform.yml

```
permissions:
  id-token: write        # OIDC — no stored AWS keys
  contents: read
  pull-requests: write   # to post the plan

- uses: aws-actions/configure-aws-credentials@v4
```

The same OIDC trick, again

GitHub Actions assumes an IAM role using a short-lived token from an OIDC trust relationship. No AWS access keys stored anywhere — exactly the pattern from the CI/CD module, now applied to infrastructure.

Nobody applies from a laptop in a mature team. The laptop is for plan; the pipeline is for apply.

LEARN THESE FROM A SLIDE, NOT FROM AN INCIDENT

Eight things that will bite you

1. **Committing terraform.tfstate**
   Secrets in git history. Add *.tfstate to .gitignore on day one.

2. **Not reading the plan**
   Every serious Terraform incident starts with someone typing yes too fast.

3. **Unpinned provider versions**
   A colleague’s init pulls a new major version and the plan fills with surprises.

4. **Hardcoding IDs and ARNs**
   Removes the dependency edge. Terraform builds things in the wrong order.

5. **Editing state by hand**
   It is JSON, it looks safe, it is not. Use the state commands.

6. **Clicking in the console “just once”**
   That change is now drift, and apply will silently revert it.

7. **Building one enormous state file**
   Slow plans, huge blast radius, and eventually nobody wants to touch it.

8. **Forgetting to destroy**
   An idle EKS control plane bills all month. Teardown is part of the lab.

Six of these eight are habits, not knowledge. Build the habits today while the stakes are ten cents.

SECURITY AND POLICY

Guardrails worth adding early

Treat state as a credential

Encrypted bucket, versioning on, a bucket policy that names exactly who can read it, and access logging. Anyone who can read state can read every secret you ever generated.

Keep secrets out of the config

Pull them at plan time from Secrets Manager or SSM. Better still, have Terraform generate them and never let a human see the value at all.

Scan before you apply

tfsec, Checkov and Trivy read your HCL and flag public buckets, open security groups and unencrypted volumes. Same Trivy you used on container images.

Encode policy as code

OPA or Sentinel can reject a plan that violates a rule — no untagged resources, no instance larger than an agreed size, no public ingress on port 22.

THE PRINCIPLE

The cheapest place to catch an infrastructure mistake is in a pull request. The most expensive is in an incident review.

THE NEXT THREE HOURS

What you are about to build

#	LAB	TIME	THE IDEA YOU TAKE AWAY
00	Setup	15m	Tooling, credentials, and how to work today
01	S3 and remote state	25m	State is the whole ballgame — including the migration
02	IAM	25m	Trust policy versus permission policy
03	EC2 and EBS	30m	Data sources, lifecycle rules, user_data
04	ECR	15m	Immutable tags and lifecycle policies
05	EKS	45m	A cluster is really just five resource types
06	Kubernetes	25m	The provider chicken-and-egg, resolved
07	Teardown	15m	Destroy order, and hunting for orphans

`region: ap-south-1` `terraform >= 1.9` `aws provider ~> 5.0` `cost: about $0.30–$0.80`

TWO RULES FOR TODAY

Everything gets your name in it, because you are sharing an account and S3 bucket names are globally unique. And nobody leaves before lab 07 finishes — an EKS control plane you forgot about bills every hour of every day.

RECAP

The five things that matter

Declarative, not imperative

Describe the destination. Let the tool work out the route.

State is the whole ballgame

It maps your names to real IDs, it holds secrets, and it must be shared and locked.

Read the plan, every time

The summary line first. Then look for anything being destroyed.

References build the graph

Never paste an ID. A reference is both a value and a dependency.

Drift is normal

Reality moves. Reconciliation is the answer, and it runs on demand.

Where this goes next

Terraform in CI with OIDC. Policy scanning in the pipeline. GitOps for the workloads. Importing infrastructure that already exists.

Everything here transfers to OpenTofu, Pulumi and CDK. You learned a discipline, not a binary.

Questions — then keyboards

Take ten minutes. Come back with terraform version printing 1.9 or newer, aws sts get-caller-identity succeeding, and kubectl on your PATH.

`terraform version` `aws sts get-caller-identity` `kubectl version --client`